

Retrieval-Augmented Generation Application — Project Report

Generative AI Fundamentals Assignment

Nagarro | 2026

GOOGLE COLAB PROJECT LINK — CLICK TO OPEN

https://colab.research.google.com/drive/1dOvrglBQC2A4O_dRPk4o-1kBD6ikuXqh#scrollTo=oXxRmTx791oO

5

Source PDFs

1,335

Text Chunks

384-dim

Embeddings

4-turn

Memory

10

Test Questions

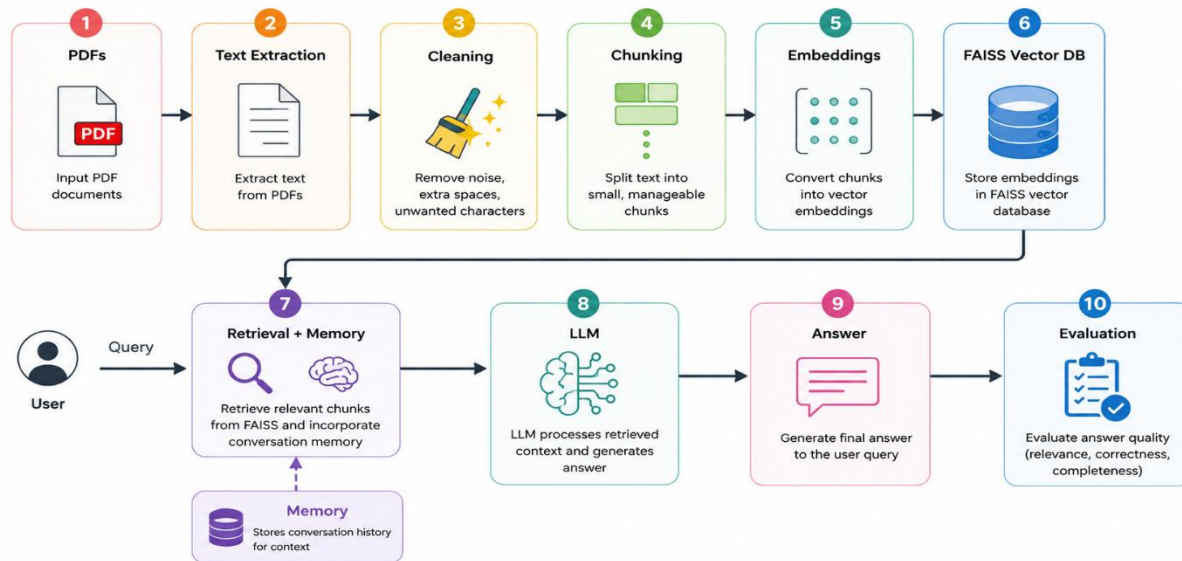
1. Overview

This project implements a Retrieval-Augmented Generation (RAG) application that ingests five foundational NLP research papers, builds a semantic vector database, and powers a conversational bot with memory of the last four interactions. The system answers user questions by retrieving relevant text from the papers and generating responses using a fully open-source language model hosted on Hugging Face.

Source Documents

- Attention Is All You Need (source PDF)
- BERT: Pre-training of Deep Bidirectional Transformers
- Language Models are Few-Shot Learners (GPT-3)
- RoBERTa: A Robustly Optimized BERT Pretraining Approach
- Exploring the Limits of Transfer Learning with T5

End-to-End Pipeline



2. Technical Implementation

2.1 PDF Ingestion & Data Sourcing

The five PDFs were downloaded programmatically from arXiv using Python's requests library with a browser-style User-Agent header to avoid request blocking. Text was extracted using the pypdf library by iterating over every page and concatenating the extracted text.

Raw text was cleaned using a regular expression to collapse excessive whitespace and line breaks into single spaces. The cleaned text was then split into chunks using RecursiveCharacterTextSplitter from Langchain-text-splitters.

```
Chunk size:      500 characters
Chunk overlap:   50 characters
Total chunks:    1,335 across all five documents
```

2.2 Vector Database Creation

Each of the 1,335 text chunks was converted into a dense 384-dimensional embedding using the all-MiniLM-L6-v2 model from sentence-transformers. These embeddings capture the semantic meaning of each chunk.

Embeddings were stored in a FAISS (faiss-cpu) index using IndexFlatL2, performing exact nearest-neighbour search using Euclidean distance. The index and chunk metadata were saved to disk (faiss_index.bin and chunks.pkl) to avoid re-embedding on every run.

```
Embedding model:    all-MiniLM-L6-v2 (sentence-transformers)
Vector dimensions:  384
FAISS index type:   IndexFlatL2 (exact Euclidean search)
Saved artefacts:    faiss_index.bin | chunks.pkl
```

2.3 Open-Source Language Model Integration

An open-source language model, **google/flan-t5-base**, was integrated from Hugging Face for answer generation. Due to a task-name incompatibility in the installed version of the transformers library, the model was loaded directly using T5Tokenizer and T5ForConditionalGeneration.

For each user query the pipeline:

- Encodes the query using the embedding model.
- Retrieves the top-k most relevant chunks from FAISS.
- Combines the retrieved chunks and the question into a structured prompt.
- Passes the prompt through the T5 model with max_new_tokens=200 and max_length=512.
- Decodes and returns the generated answer.

2.4 Conversational Bot with Memory

The bot maintains a conversation memory list capped at the last 4 turns, satisfying the assignment requirement exactly. Each turn stores the question and the generated answer as a dictionary.

Before each response, stored turns are formatted into a readable history block and injected into the prompt alongside retrieved context. After generating an answer, the new turn is appended. If memory exceeds 4 turns, the oldest entry is removed using list.pop(0).

```
Memory limit:  4 turns (8 messages: 4 user + 4 assistant)
Storage:       Python list of {question: ..., answer: ...}
Enforcement:   list.pop(0) when len(memory) > 4
```

3. Evaluation

3.1 Test Questions & Ground Truth

#	Question	Ground Truth Reference
1	What is the main innovation in Attention Is All You Need?	The Transformer architecture, relying entirely on self-attention, removing recurrence.
2	What does the self-attention mechanism do?	It lets each word attend to all others to capture relationships and context.
3	What does BERT stand for?	Bidirectional Encoder Representations from Transformers.
4	How is BERT trained?	Using masked language modelling and next sentence prediction on large unlabelled text.

#	Question	Ground Truth Reference
5	What is GPT-3 known for?	A 175-billion-parameter model capable of few-shot learning without fine-tuning.
6	What is few-shot learning in GPT-3?	Performing tasks given only a few prompt examples, without updating model weights.
7	How does RoBERTa improve on BERT?	More data, longer training, larger batches, no NSP, and dynamic masking.
8	What is the main idea of T5?	Treating every NLP task as a text-to-text problem.
9	Why is the Transformer faster to train than RNNs?	Self-attention allows parallel processing, unlike sequential RNNs.
10	What do these five papers have in common?	All are foundational Transformer-based NLP models.

3.2 Evaluation Methodology

A self-devised evaluation metric based on cosine similarity of sentence embeddings was used, as permitted by the assignment. The same all-MiniLM-L6-v2 model used for retrieval ensures consistency. Two scores were computed per question:

- Answer Relevance — cosine similarity between the bot's answer embedding and the ground-truth embedding. Measures accuracy.
- Faithfulness — cosine similarity between the bot's answer embedding and the combined retrieved context embedding. Measures whether answers are grounded in source documents rather than hallucinated.

4. Results & Discussion

4.1 Evaluation Scores

Q#	Answer Relevance	Faithfulness
1	0.250	0.629
2	0.169	0.144
3	0.236	0.476
4	0.220	0.590
5	0.413	0.506
6	0.252	0.689
7	0.270	0.526
8	0.126	0.614
9	0.071	0.195
10	0.158	0.132
Average	0.216	0.450

4.2 Discussion

Faithfulness averaged 0.450, indicating the bot's answers were reasonably grounded in retrieved source content rather than fabricated. Answer relevance averaged 0.216, which was lower for two primary reasons:

- The flan-t5-base model produces short, concise answers. Cosine similarity penalises correct but briefly phrased answers when compared against fuller ground-truth references.
- The metric compares against a single reference phrasing. A correct answer worded differently still scores low.

Questions 9 and 10, which required cross-paper synthesis, scored lowest because FAISS retrieves narrow, specific chunks rather than broad summaries.

4.3 Challenges Faced

- A transformers version incompatibility caused text2text-generation to be unrecognised in pipeline(). Resolved by loading T5Tokenizer and T5ForConditionalGeneration directly.
- The langchain.text_splitter import path changed in newer versions. Resolved by installing and importing from langchain-text-splitters.
- Balancing prompt length (context + memory + question) against the model's 512-token input limit required careful truncation settings.

5. Conclusion

This project successfully delivered a complete, fully open-source RAG pipeline. Five NLP research papers were ingested, cleaned, and split into 1,335 chunks. Embeddings were stored in a FAISS vector database enabling fast semantic retrieval. An open-source Hugging Face language model generated answers grounded in retrieved context. A 4-turn conversational memory enabled coherent follow-up interactions.

Evaluation across 10 questions demonstrated that the system retrieves faithfully from source documents (average faithfulness 0.450), with answer relevance (0.216) limited primarily by model size and metric strictness. The project provides a strong foundation for further improvement through larger models and richer evaluation frameworks.

6. Project Link

The complete implementation is available in the Google Collab notebook below. All code — ingestion, embedding, retrieval, LLM integration, memory management, and evaluation — is contained in a single executable notebook.

GOOGLE COLAB NOTEBOOK — CLICK TO OPEN

https://colab.research.google.com/drive/1dOvrglBQC2A4O_dRPk4o-1kBD6ikuXqh#scrollTo=oXxRmTx791oO
